

Evaluation Complete

Your evaluation results are ready



Candidate Name

nitin.bharti@irissoftware.com



Interview Date

22-Jul-2026



Overall Score

66%



Evaluation Duration

134.6 min



Category Scores

Data Structures & Algorithms

43%

JUnit

90%

Concurrency & Multithreading

77%

Spring

85%

Programming(java)	71%
Microservices	0%
JVM	100%
Database	85%
Core Java	95%
Spring Boot	0%
Collections	100%
OOPS	0%
Generics	95%
API Design	63%
Java 8+	100%
Memory Management	63%
Garbage Collection	100%
Design Patterns	100%
Spring Boot JPA	0%

Strengths



Basic understanding of Java memory management concepts

- ✓ Correctly uses @CreateDate and @LastModifiedDate annotations
- ✓ Clear explanation of the project's architecture and technical decisions
- ✓ Implementing a BFS algorithm to find the shortest sequence of moves
- ✓ Provides clear explanation of method chaining and build method

Areas for Improvement

- ⚠ Might benefit from clarifying the exact implications of reduced critical section size on concurrency
- ⚠ Missed discussing benefits of using Builder pattern, such as reduced constructor complexity and improved readability
- ⚠ Be more precise with technical terms (e.g., 'J2W' instead of 'Java')
- ⚠ Naming conventions (e.g., `bq` instead of `queue`)
- ⚠ Candidate did not attempt to solve the problem - template code was left unchanged

Detailed Question Feedback

Q1

Project Experience

medium

N/A

Please explain your most recent project in detail. Describe the technologies used, your role and responsibilities, the challenges you faced, and how you overcame them. Also explain the architecture and any significant technical decisions you made.

Candidate's Answer:

Okay, so hi Nick in the site. So in the previous project, the roles and responsibility. So basically, I am working as a team with under me. I mean, there are around seven to eight developers. So I'm working. So basically, the dynamics project that I work in is purely a development task and we start working from the scratch. So we have developed the spring boot application. And I mean, the team members are also there from the Angular. So my role is basically to begin the team to deliver whatever specific I mean, the story that we have to deliver as part of the current sprint, right? So sprint are basically for three weeks. And whatever stories I have been picked, we need to deliver end to end within those three weeks. And then we have to Deploy those stories in the production environment. So I am responsible for making sure whatever stories that have been picked in the current split and need to be delivered end to end whatever developers or the developers are also there. So I have to make sure if they're facing any challenges, I need to make sure that should be fixed. I mean, whether it's an infrastructure challenge, development challenge, or any other challenge. So the previous project functionality. So I was, the project is basically a dynamics project that is basically a Mexico retail unit that is basically moving out from the city. So we have to making a standalone services for the project. And that is basically a combination of front end and back end.

In front end, we are having an Angular portal for the dynamics that user or the client can log in on to it and they can perform a different functionality over there. So main feature are basically for the portal is we are having the notification enabled. And we as an portal, right, so we are having a different functionality, like the app dynamics features we integrated, we are having the functionality like the roles and admin roles and whatever roles and the user can have and we can have the functionality like recently, new page and favorites. And there are so many things that are basically part of that project. So the architecture basically on the architecture level, so we are having the Angular application as a UI. So the user can hit from the portal URL. So portal URL is basically is linked with the load balancer to whatever virtual VMs we are having right. So that is basically configured by the infoting. And once the server is being hit, the request will be go to the back end and via the load balancer only it will be picked the proper microservice, whatever service is being triggered from the front end, it will connect with the database or whatever request is being made, that is being hit. So and also in parallel, we are having a scheduled up also so that is being triggered by autosets and those autosets basically depending on the requirement, it is daily, weekly, monthly and that is being triggered and load the data from the files depending on the requirements and they will generate those Excel sheets, SL and the different type of sheets. The design patterns that we have used is basically the functional, the observer design patterns we have implemented in the project along with that we use builder pattern as well. The domain right so as I told domain is basically the that is in this standalone form right so we have used the still set like Java, microservices, autosets, DevOps, we are using Jenkins and all those things at light speed and hardness will be used for the deployment purpose. So the list of technologies, frameworks and tools so we are having the, so we are having the, we are having the J2W, we are using Java 17, we are using along with that we are using the tools like the hardness, light speed autosets, we use the such tools for postman for hitting the endpoint intelligently.

Good: Clear explanation of the project's architecture and technical decisions, Good understanding of microservices and load balancing concepts

Improve: Be more precise with technical terms (e.g., 'J2W' instead of 'Java'), Provide more details about the project's functionality, such as how the notification system works or what specific features were implemented

Q2

Data Structures & Algorithms

hard

N/A

85%

Tile Board Puzzle

You are given a board of Red(R) and Black(B) tiles. You may swap any two adjacent tiles (horizontally or vertically) to transform the board from a starting state to an ending state.

Given a starting board and an ending board, write a function that returns one possible sequence of moves to transform the start into the end. Each move should specify the two positions being swapped.

Example:

```
start = [
  ["R", "R", "R"],
  ["B", "B", "B"]
]
```

```
end = [
  ["B", "R", "R"],
  ["R", "B", "B"]
]
```

One possible sequence:

Swap (0,0) with (1,0): `[["B","R","R"],["R","B","B"]]` => matches end

`findMoves(start, end) => [[(0,0), (1,0)]]`

Additional Test Case:

```
start2 = [
  ["R", "B"],
  ["B", "R"]
]
```

```
end2 = [
  ["B", "R"],
  ["R", "B"]
]
```

`findMoves(start2, end2) => [[(0,0),(0,1)], [(1,0),(1,1)]]` or `[[ (0,0),(1,0)], [(0,1),(1,1)]]`
(Any valid sequence of adjacent swaps is acceptable)

Complexity variables:

r: number of rows

c: number of columns

Candidate's Code Answer:

```
import java.util.*;

public class TileBoardPuzzle {

    /**
     * Given a starting and ending board configuration of R/B tiles,
     * return a sequence of adjacent tile swaps to transform start into end.
     *
     * @param start 2D array of the starting board
     * @param end 2D array of the ending board
     * @return List of swap pairs, each pair is [(r1,c1), (r2,c2)]
     */
    public static List<int[][]> findMoves(String[][] start, String[][] end) {
        // Write your code here
        int r= start.length,c =start[0].length;
        java.util.function.Function<String[][], String> serialize = b ->{
            StringBuilder sb=new StringBuilder();
            for(String[] row : b)
                for (String cell:row) sb.append(cell);
            return sb.toString();
        };

        String target= serialize.apply(end);
        Queue<String[][]> bq=new LinkedList<>();
        Queue<List<int[][]>> pq =new LinkedList<>();
        Set<String> visited=new HashSet<>();

        bq.add(start);
        pq.add(new ArrayList<>());
        visited.add(serialize.apply(start));
        int[][] dirs={{0,1},{1,0}};
```

```

while(!bq.isEmpty()){
    String[][] board=bq.poll();
    List<int [][]> path= pq.poll();

    if (serialize.apply(board).equals(target))
        return path;
    for(int i=0;i<r;i++){
        for(int j=0;j<c;j++){
            for(int[] d : dirs){
                int ni=i+d[0], nj=j+d[1];
                if(ni<r &&nj<c && !board[i][j].equals(board[ni][nj])){
                    String[][] next=new String[r][c];
                    for(int k=0;k<r;k++)
                        next[k]=board[k].clone();

                    String temp=next[i][j];
                    next[i][j]=next[ni][nj];
                    next[ni][nj]=temp;

                    String key=serialize.apply(next);

                    if(visited.add(key)){
                        List<int[][]> newPath=new ArrayList<>(path);
                        newPath.add(new int[][]{{i,j},{ni,nj}});
                        bq.add(next);
                        pq.add(newPath);
                    }
                }
            }
        }
    }

    return new ArrayList<>();
}

public static void main(String[] args) {
    String[][] start1 = {{ "R", "R", "R"}, { "B", "B", "B"} };
    String[][] end1 = {{ "B", "R", "R"}, { "R", "B", "B"} };
    System.out.println(findMoves(start1, end1));
    // Expected: [[(0,0),(1,0)]]

    String[][] start2 = {{ "R", "B"}, { "B", "R"} };
    String[][] end2 = {{ "B", "R"}, { "R", "B"} };
    System.out.println(findMoves(start2, end2));
    // Expected: any valid sequence of adjacent swaps
}

```

Good: Implementing a BFS algorithm to find the shortest sequence of moves, Handling both horizontal and vertical swaps

Improve: Naming conventions (e.g., `bq` instead of `queue`), Variable names (e.g., `next` instead of `newBoard`)

Q3

JUnit

hard

N/A

90%

Write tests for CompletableFuture and async methods.

Candidate's Code Answer:

```

import org.junit.jupiter.api.Test;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.TimeoutException;

public class AsyncServiceTest{

    public CompletableFuture<String> fetchDataAsync() {
        return CompletableFuture.supplyAsync(()-> "Hi, There!!");
    }

    public CompletableFuture fetchDataErrorAsync(){
        return CompletableFuture.failedFuture(new Exception("Exception Async
Error"));
    }

    public CompletableFuture<String> fethcSlowDataAsync(){
        return CompletableFuture.supplyAsync(()->{
            try{
                Thread.sleep(2000);
            } catch(InterruptedException e){
                Thread.currentThread().interrupt();
            }
            return "Result is Delayed";
        });
    }

    @Test
    public void testFetchDataAsync_success() throws Exception{
        CompletableFuture<String> future= fetchDataAsync();
        String result = future.get(2,TimeUnit.SECONDS);
        assertEquals("Hi, There!!",result);
        assertTrue(future.isDone());
        assertFalse(future.isCompletedExceptionally());
    }

}

```

Good: Properly uses CompletableFuture to handle asynchronous operations, Tests various scenarios including success, error, and slow data

Improve: Add testing for edge cases such as timeout or cancellation, Consider using more descriptive variable names

Q4

Concurrency & Multithreading

hard

N/A

83%

What is the difference between synchronized block and synchronized method?

Candidate's Answer:

The main difference between a synchronized block and a synchronized method are basically around the scope, locking and granularity and the performance of the code. So if we talk about these scopes and granularity, so the synchronized methods lock the entire object using this for instance or the class object for a static method. On the other hand, the synchronized block allows you to lock only a specific section of a code and you can explicitly choose which object monitor to lock on. This is for the scope and if we talk about the performance, the synchronized block offers better

performance and concurrency because they reduce the critical section size. So on the specific lines inside the block are locked whereas the synchronized methods lock entire code inside it and increasing the content in the wait time for the other thread. And if we talk about the flexibility to synchronize blocks give more final control letting the scope synchronize and precisely to share the resource being modified whether the method level locking is an all or nothing approach.

Good: Correctly identified the main differences between synchronized blocks and methods, Provided a clear explanation of scope, locking, granularity, and performance

Improve: Could have provided more specific scenarios where each is used, Might benefit from clarifying the exact implications of reduced critical section size on concurrency

Q5

Spring

medium

N/A

85%

Write code for auditing entities using @CreateDate and @LastModifiedDate.

Candidate's Code Answer:

```
@Configuration
@EnableJpaAuditing
class JpaAuditingConfig{

}
@Entity
@EntityListeners(AuditingEntityListener.class)
public class User{

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long id;

    private String name;

    @CreateDate
    @Column(name="created_date", nullable=false,updateable=false)
    private LocalDateTime createdDate;

    @LastModifiedDate
    @Column(name="last_Modified_Date")
    private LocalDateTime lastModifiedDate;

    public User(){}
    public User(String name){
        this.name=name;
    }

}
```

Good: Uses @EnableJpaAuditing to enable auditing for entities, Correctly uses @CreateDate and @LastModifiedDate annotations

Improve: Fix typo in GenerationType.IDENTITY annotation, Correct column name for lastModifiedDate

Q6

Programming(java)

hard

N/A

71%

What is the Builder pattern? When would you use it over constructor?

Candidate's Answer:

So, the builder design pattern is a creational design pattern, it's I think, I mean, it's part of it and used to construct the complex objects step by step. So instead of passing a bunch of arguments into a huge constructor, with this pattern we can set only the fields we need using method chaining. And then we can call the build method at the end to get the final object out of it. So there are different criteria in which we should go with the builder design pattern. The first one, if we are having too many parameters when an object has four or five or more parameters, construction have become messy and hard to read. So in that case, we should go with the builder design pattern. When we are having optional fields, so if many of fields are there, which are optional, so construction force is to pass the null value when we are calling that constructor. So in that case also, it's better to go with the builder pattern, because writing a bunch of overloaded constructor, that is quite hectic. And it is quite readable and more safe if you are going with the builder pattern.

Good: Correctly identifies the Builder pattern as a creational design pattern, Provides clear explanation of method chaining and build method

Improve: Could have provided more specific examples or scenarios where Builder pattern is preferred over constructors, Missed discussing benefits of using Builder pattern, such as reduced constructor complexity and improved readability

Q7

Microservices

medium

N/A

0%

Explain the Strangler Fig pattern for microservices migration.

Improve: Candidate did not provide an answer

Q8

JVM

easy

N/A

100%

Which statement about intrinsic locks (monitor locks) is true?

Candidate's Answer:

A synchronized static method locks the class object

Good: Correct answer selected

Q9

Database

medium

N/A

85%

Design a schema for an e-commerce system with proper constraints.

Candidate's Code Answer:

```
CREATE TABLE users(  
    user_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    email VARCHAR2(255) NOT NULL UNIQUE,  
    password_hash VARCHAR2(255) NOT NULL,  
    first_name VARCHAR2(100) NOT NULL,  
    last_name VARCHAR2(100) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL  
);  
  
CREATE TABLE products(  
    product_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    product_name VARCHAR2(255) NOT NULL,  
    price NUMBER(10,2) NOT NULL,  
    stock_quantity NUMBER(10) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL  
);
```

```

product_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
last_name VARCHAR2(255) NOT NULL,
description CLOB,
price NUMBER (10,2) NOT NULL CHECK(price>=0).
stock_quantity NUMBER(10) DAFAULT 0 NOT NULL CHECK(stock_quantity >=0),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);

CREATE TABLE ORDER(
    order_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id NUMBER NOT NULL,
    total_amount NUMBER(10,2) NOT NULL CHECK(total_amount>=0),
    status VARCHAR2(50) NOT NULL CHECK( status IN
('SHIPPED','PENDING','PAID','DELIVERED','CANCELLED')),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT fk_orders_users FOREIGN KEY (user_id) REFERENCES users(user_id)
ON DELETE CASCADE
);

CREATE TABLE ORDERS_ITEMS(
    order_item_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    order_id NUMBER NOT NULL,
    product_id NUMBER NOT NULL,
    quantity NUMBER(10) NOT NULL CHECK(quantity>0),
    price_at_purchase NUMBER(10,2) NOT NULL CHECK (price_at_purchase>0),
    CONSTRAINT fk_items_order FOREIGN KEY (order_id) REFERENCES orders(order_id)
ON DELETE CASCADE,
    CONSTRAINT fk_items_product FOREIGN KEY(product_id) REFERENCES
products(product_id),
    CONSTRAINT uq_order_product UNIQUE(order_id,product_id)
);

CREATE TABLE PAYMENTS(
    payment_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    order_id NUMBER NOT NULL UNIQUE,
    payment_methods VARCHAR2(50) NOT NULL CHECK(payment_methods IN ('CREDIT-
CARD','UPI','COD')),
    payment_status VARCHAR2(50) NOT NULL CHECK (payment_status IN
('PENDING','COMPLETED','FAILED','REFUNDED')),
    transaction_id VARCHAR2(255) UNIQUE,
    amount NUMBER(10,2) NOT NULL CHECK (amount>=0),
    paid_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_payments_order FOREIGN KEY (order_id) REFERENCES
orders(order_id) ON DELETE CASCADE
)

```

Good: Correctly defines the relationships between tables, Uses proper data types for each column

Improve: Rename 'last_name' in 'products' table to 'product_name', Add a check constraint to allow zero values for 'price' in 'products' table

Q10

Core Java

easy

N/A

95%

Write a program to convert a List to Map using Streams handling duplicate keys.

Candidate's Code Answer:

```
import java.util.*;
import java.util.stream.Collectors;
public class ListToMapTest{

    static class Item{
        private int id;
        private String name;

        public Item(int id, String name){
            this.id=id;
            this.name=name;
        }

        public int getId(){
            return id;
        }

        public String getName(){
            return name;
        }

        @Override
        public String toString(){
            return name;
        }
    }

    public static void main(String[] args){
        List<Item> itemList= Arrays.asList(
            new Item(1,"Apple"),
            new Item(2,"Banana"),
            new Item(1,"Grapes")
        );
        Map<Integer,String> itemMap=itemList.stream()
            .collect(Collectors.toMap(
                Item::getId,
                Item::getName,
                (existingValue,newValue) ->newValue
            ));
        System.out.println(itemMap);
    }
}
```

Good: Handles duplicate keys correctly, Uses Stream API for efficient data processing

Improve: Consider using a more descriptive variable name instead of 'itemMap', Code could benefit from additional edge case handling, such as empty input lists or null elements

Q11 Spring Boot hard N/A

0%

Implement custom health indicators for Spring Boot Actuator.

Improve: Candidate did not provide an answer

Q12 Collections easy N/A

100%

Which preserves insertion order?

Candidate's Answer:

LinkedHashSet

Good: Correct answer selected

Q13

OOPS

medium

N/A

0%

Toll Booth Log Analysis -- Fix the Bug

You are building software to analyze logs from toll booths on a highway. The highway is a divided highway with limited access -- the only way to get on or off is through a toll booth.

There are three types of toll booths:

- ENTRY (E): A car enters the highway.
- EXIT (X): A car leaves the highway.
- MAINROAD (M): Sensors record a license plate at full speed.

Each log line format: "34400.409 SXY288 210E ENTRY"

- 34400.409 = timestamp in seconds
- SXY288 = license plate
- 210E = location (210 km) and direction (E=east-bound)
- ENTRY = booth type (ENTRY, EXIT, or MAINROAD)

The code below has a bug that causes the tests to fail. The tests are correct (treat them as the specification). Find the bug, and fix it

Test expectations:

logEntry.getTimestamp() should return 44776.619f (float)

logEntry.getLicensePlate() should return "KTB918"

logEntry.getLocation() should return 310

logEntry.getDirection() should return "EAST"

logEntry.getBoothType() should return "MAINROAD"

Candidate's Code Answer:

```
import static org.junit.Assert.assertEquals;
import java.util.*;
```

```
class LogEntry {
    private final String timestamp;
    private final String licensePlate;
    private final String boothType;
    private final int location;
    private final String direction;

    public LogEntry(String logLine) {
        String[] tokens = logLine.split(" ");
        this.timestamp = tokens[0];
        this.licensePlate = tokens[1];
```

```

        this.boothType = tokens[3];
        this.location =
            Integer.parseInt(tokens[2].substring(0, tokens[2].length() - 1));
        String directionLetter = tokens[2].substring(tokens[2].length() - 1);
        if (directionLetter.equals("E")) {
            this.direction = "EAST";
        } else if (directionLetter.equals("W")) {
            this.direction = "WEST";
        } else {
            throw new IllegalArgumentException();
        }
    }

    public float getTimestamp() { return timestamp; }
    public String getLicensePlate() { return licensePlate; }
    public String getBoothType() { return boothType; }
    public int getLocation() { return location; }
    public String getDirection() { return direction; }
}

// Tests (correct -- treat as spec):
// LogEntry e1 = new LogEntry("44776.619 KTB918 310E MAINROAD");
// assertEquals(44776.619f, e1.getTimestamp(), 0.0001);
// assertEquals("KTb918", e1.getLicensePlate());
// assertEquals(310, e1.getLocation());
// assertEquals("EAST", e1.getDirection());
// assertEquals("MAINROAD", e1.getBoothType());

```

Improve: Candidate did not attempt to solve the problem - template code was left unchanged

Q14

Generics

medium

N/A

95%

Write Generic Utility Methods Using Bounded Wildcards

Complete the generic utility methods below by replacing each ??? with the correct bounded wildcard type. Follow the PECS principle (Producer Extends, Consumer Super).

The Animal class hierarchy is: Animal -> Dog -> GoldenRetriever and Animal -> Cat.

For each method, determine whether the list parameter is a producer (read from) or consumer (write to), and choose the appropriate wildcard type accordingly. Then implement the method body.

Candidate's Code Answer:

```

import java.util.*;

class Animal {
    String name;
    Animal(String name) { this.name = name; }
    public String toString() { return name; }
}

class Dog extends Animal {
    Dog(String name) { super(name); }
}

```

```

class GoldenRetriever extends Dog {
    GoldenRetriever(String name) { super(name); }
}
class Cat extends Animal {
    Cat(String name) { super(name); }
}

public class WildcardExercise {

    // 1. Prints all animals in the list.
    public static void printAnimals(List<? extends Animal> animals) {
        // Write your code here
        for(Animal a: animals)
            System.out.println(a);
    }

    // 2. Adds a Dog to the given list.
    public static void addDog(List<? super Dog> list, Dog dog) {
        // Write your code here
        list.add(dog);
    }

    // 3. Copies all elements from source to destination.
    public static <T> void copyAll(List<? extends T> source, List<? super T>
destination) {
        // Write your code here
        for(T item: source){
            destination.add(item);
        }
    }

    // 4. Returns the animal with the longest name from a list.
    public static Animal longestName(List<? extends Animal> animals) {
        // Write your code here
        if(animals==null || animals.isEmpty()){
            return null;
        }
        Animal longest=animals.get(0);
        for(Animal a : animals){
            if(a.name.length()>longest.name.length()){
                longest=a;
            }
        }
        return longest;
    }

    public static void main(String[] args) {
        List<Dog> dogs = Arrays.asList(new Dog("Rex"), new
GoldenRetriever("Buddy"));
        printAnimals(dogs); // Should work

        List<Animal> animals = new ArrayList<>();
        addDog(animals, new Dog("Max")); // Should work

        List<Animal> dest = new ArrayList<>();
        copyAll(dogs, dest); // Should work
        System.out.println(dest);
    }
}

```

Good: Proper use of bounded wildcards, Correct implementation of printAnimals method, Correct implementation of addDog method, Correct implementation of copyAll method, Correct implementation of longestName method

Improve: Consider using more descriptive variable names, Consider adding null checks for the input lists

Q15

API Design

easy

N/A

63%

When is REST better than Message-based

Candidate's Answer:

So REST is better than the message-based architecture in a scenario where you need a synchronous response communication between client and server. So REST is basically ideal when we are having a real-time or immediate that needed when you have to do a simple cloud operation for basic create, create, update, and delete. Then we have to do the standard REST protocol and format response like JSON, making it easier for external client. We have also inferred part integrations to interact with the system without meeting the specific local library or the complex structure.

Good: correctly identifies scenarios where REST excels, mentions standardizing REST protocol and format responses

Improve: provide a clear explanation for why synchronous response communication makes REST more suitable, compare REST with message-based architecture in terms of scalability, complexity, or other relevant factors

Q16

Java 8+

easy

N/A

100%

Which is short circuiting?

Candidate's Answer:

findAny

Good: Correct answer selected

Q17

Memory Management

easy

N/A

63%

What is the difference between Heap and Stack memory?

Candidate's Answer:

java comes when we have to do what they store, their life cycle and how threads access them. So stack memory holds short field execution data like method calls primitive local variables and the references to object whereas the heap memory on the other hand stores all actual java objects and instance variables graded using a new keyword. So stack memory is thread private each thread has its own local stack making stack access completely thread safe whereas heap memory shared across all the threads in the application which is why shared state on the heap often requires synchronization.

Good: Correctly identifies key differences between Heap and Stack memory, Basic understanding of Java memory management concepts

Improve: Provide a more detailed explanation of Stack memory's thread-safety features, including how each thread has its own local stack., Clarify the distinction between Heap and Stack memory in terms of object lifetime, emphasizing that objects stored on the Heap can be accessed by multiple threads.

Q18

Garbage Collection

easy

N/A

100%

What is the default garbage collector in Java 11?

Candidate's Answer:

G1 GC

Good: Correct answer selected

Q19

Design Patterns

easy

N/A

100%

The Saga pattern addresses:

Candidate's Answer:

Distributed transactions via a sequence of local transactions and compensations

Good: Correct answer selected

Q20

Spring Boot JPA

easy

N/A

0%

Implement Pagination and Sorting for a Product Listing in Spring Boot

You need to build a product listing API that supports pagination and sorting using Spring Data JPA. Complete the code skeleton below to:

1. Define the JPA entity with proper annotations.
2. Create a repository interface that supports pagination.
3. Implement the service method that accepts page number, page size, and sort field.
4. Implement the controller endpoint that takes pagination parameters from the request.

The API should be called like: GET /api/products?page=0&size=10&sort=price,desc

The response should include: content (list of products), totalPages, totalElements, currentPage.

Candidate's Code Answer:

```
import org.springframework.data.domain.*;
import org.springframework.data.jpa.repository.*;
import org.springframework.web.bind.annotation.*;
```

```

import javax.persistence.*;
import java.util.*;

// 1. Complete the JPA Entity
@Entity
@Table(name = "products")
class Product {
    // Write your code here -- add fields: id, name, price, category
    // with proper JPA annotations (@Id, @GeneratedValue, etc.)
}

// 2. Complete the Repository interface with pagination support
interface ProductRepository {
    // Write your code here -- extend the correct Spring Data interface
}

// 3. Complete the Service method
@Service
class ProductService {
    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    public Page<Product> getProducts(int page, int size, String sortField,
String sortDirection) {
        // Write your code here:
        // - Create a Sort object from sortField and sortDirection
        // - Create a Pageable object with page, size, and sort
        // - Query the repository with the Pageable
        return null;
    }
}

// 4. Complete the Controller endpoint
@RestController
@RequestMapping("/api/products")
class ProductController {
    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping
    public ResponseEntity<?> getProducts(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "10") int size,
        @RequestParam(defaultValue = "name") String sort,
        @RequestParam(defaultValue = "asc") String direction) {
        // Write your code here:
        // - Call service to get paginated results
        // - Return response with content, totalPages, totalElements,
currentPage
        return null;
    }
}

```

Improve: Candidate did not attempt to solve the problem - template code was left unchanged

Q21

Data Structures & Algorithms

medium

N/A

0%

Given a keyword and a string, bold all occurrences of the keyword by wrapping them in `<b>` tags.

Part 1:

```
function highlight(keyword: string, text: string): string;
```

// Example 1:

Input: keyword = "fig", text = "figma"

Output: "figma"

Followup 1:

If there are overlapping matches, the bold tags should cover the maximum possible characters.

// Example 2:

Input: keyword = "ana", text = "banana"

Output: "banana"

// NOT "banana" - we want to bold the maximum coverage

Followup 2:

How would you handle multiple keywords?

Improve: Candidate did not provide an answer

Q22

Concurrency & Multithreading

medium

N/A

71%

What is the difference between `submit()` and `execute()` in a `ThreadPoolExecutor`?

Candidate's Answer:

So execute treated void making it fire in forward method that means once we fired this method call we need to it will forget and it will because it will not return anything whereas submit returns a future object which allow you to retrieve the result of the task for check its completion status using method like get. Next is execute only accept a enable task meaning cannot take cannot return a value directly whereas submit is overloaded it can accept both the neighbor and callable task allowing it to return computed results.

Good: Correctly identifies key differences between `submit()` and `execute()`, Good understanding of Future objects and their usage

Improve: Clarify task acceptance for both `execute()` and `submit()`: `execute()` only accepts callable tasks, whereas `submit()` can accept both callable and neighbor tasks, Provide a clear explanation of the implications of using `execute()` versus `submit()`

 [Download Report \(PDF\)](#)

Evalo - AI-Powered Technical Evaluation Platform